МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине **«Программное обеспечение мехатронных и робототехнических систем»**

Тема работы: «Программное обеспечение системы управления мехатронного модуля управления защитной дверью»

Студенты группы: КРБО-02-23 Евлампиев И. С. _____
Мальчуковский М. О. _____
Проклин В. С. _____
Филатов Г. А. _____
Преподаватель: Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

1 Введение

Программное обеспечение промышленных систем управления электроприводами строится из типовых модулей, каждый из которых решает отдельную подзадачу: управление исполнительным устройством, обработку сигналов датчиков, индикацию состояния. Такое разделение упрощает отладку и позволяет повторно использовать готовые модули в других проектах.

В лабораторной работе этот подход применяется к приводу защитной двери. Логика управления описывается конечными автоматами и оформляется в виде функциональных блоков на языке ANSI C в среде B&R Automation Studio версии 4.5.2. Конечный автомат хорошо подходит для описания двери, поскольку поведение привода определяется текущим состоянием (дверь открыта, закрыта, разгоняется, движется, замедляется) и событиями (срабатывание датчика положения, смена направления движения).

2 Цель работы

Получение навыков построения программного обеспечения промышленных систем управления на базе функциональных блоков и конечных автоматов на примере системы управления приводом защитной двери.

3 Исходные данные

Разработать программное обеспечение системы автоматического управления приводом защитной двери (рисунок 1). Дверь оснащена асинхронным двигателем с частотным преобразователем и четырьмя датчиками положения S0–S3, которые реагируют на пластину, закреплённую на двери. Открытие и закрытие двери задаётся тумблером направления: значение 0 соответствует открытию, значение 1 закрытию. Датчик S0 расположен у открытого положения, датчик S3 у закрытого.

При включении системы дверь должна двигаться в заданную сторону на малой скорости, чтобы определить своё местоположение, при этом индикаторы мигают с частотой 1 Гц. После этого происходит переход в рабочий режим: на участке S1–S2 обеспечивается максимальная скорость, на участках S0–S1 и S2–S3 малая, для безопасности движения. Индикаторы показывают положение двери. Система управления должна быть выполнена в виде функционального блока, допускающего повторное использование.

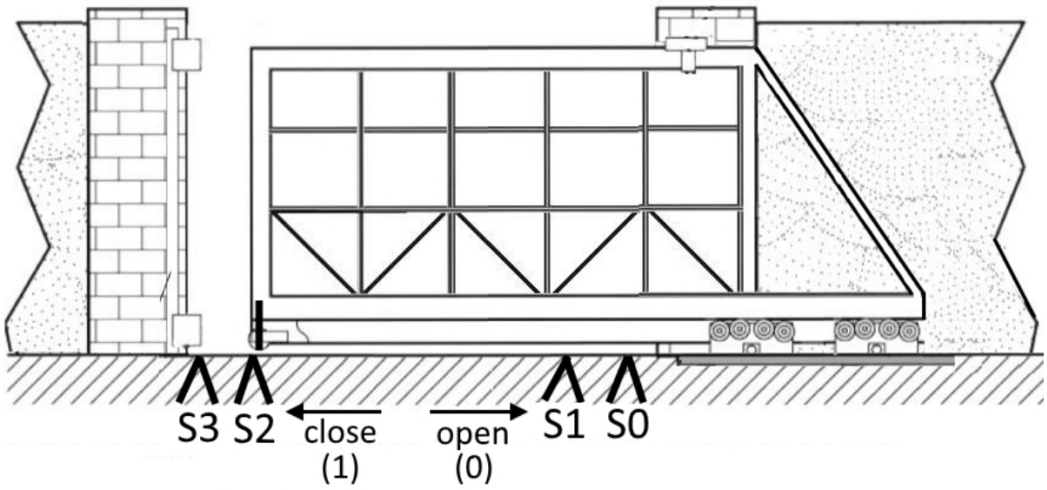


Рис. 1 Устройство защитной двери

Параметры асинхронного двигателя, заданные в конфигурации частотного преобразователя по паспортным данным, приведены в таблице 1.

Таблица 1 – Параметры асинхронного двигателя

Параметр	**Название в конфигурации преобразователя**	**Значение**
Номинальное напряжение	UNS Rated Motor volt	220
Номинальная частота	FRS Rated motor freq	500
Сопротивление статора	RSC Cold Stator resist	33000
Коэффициент мощности	COS Motor 1 Cosinus Phi	64
Номинальная скорость	NSP Rated motor speed	890
Номинальный ток	NCR Rated motor current	10

Заданные скорости движения двери выбраны относительно номинальной скорости двигателя: малая скорость 200 об/мин, максимальная 850 об/мин. Их доля от номинальной скорости определяется по формуле (1):

$$n_{\text{отн}} = \frac{n}{n_{\text{ном}}} \cdot 100\,\% \tag{1}$$

где n – заданная скорость вращения, об/мин; $n_{\text{ном}}$ – номинальная скорость вращения двигателя, равная 890 об/мин. Для малой скорости получается 22,5 %, для максимальной 95,5 % от номинальной. Знак заданной скорости определяет направление вращения: при открытии скорость положительная, при закрытии отрицательная.

4 Ход работы

4.1 Описание действий и использованного аппарата

Работа выполнена в среде B&R Automation Studio версии 4.5.2.102 на учебном стенде. В состав стенда входят программируемый логический контроллер 4PP065_0571_P74, модуль цифровых входов X20DI9371 (тумблер направления и датчики положения), модуль цифровых выходов X20DO9322 (светодиодные индикаторы), частотный преобразователь ACOPOSinverter (модуль 8I64xxxxxxx.00x-1) и асинхронный двигатель привода двери.

Работа проводилась в следующей последовательности. В исходной конфигурации оборудования к имеющимся модулям добавлен модуль частотного преобразователя и присоединён к шине X2X. В конфигурации преобразователя заданы параметры двигателя по таблице 1. Регистры ETAD, ETI, CMDD и CMI переведены из битового представления в представление словами (packed), чтобы состояние преобразователя читалось, а команды управления и заданная скорость передавались одной переменной. Затем создана программа Main на ANSI C и библиотека Library с тремя функциональными блоками, выполнен мэппинг переменных на каналы модулей и преобразователя (подраздел 4.6). Проект скомпилирован без ошибок, загружен на контроллер через сетевое подключение (Online Install), после чего контроллер перешёл в режим RUN. Проверка работы выполнена в режиме онлайн в окне Watch.

4.2 Структурные схемы конечных автоматов

Логика двери описана конечным автоматом из десяти состояний (рисунок 2). После включения автомат находится в состоянии ST_INIT и ожидает, пока преобразователь перейдёт в рабочее состояние. Затем он переходит в ST_UNKNOWN: дверь движется с малой скоростью в направлении, заданном тумблером, а индикаторы мигают. Движение продолжается до срабатывания датчика конечного положения: S0 при открытии или S3 при закрытии. После этого автомат переходит в ST_OPEN или ST_CLOSE, положение двери определено, и система находится в рабочем режиме.

Из состояния ST_OPEN при команде закрытия дверь движется по цепочке ST_ACC_POS, ST_POS, ST_DEC_POS (переходы по датчикам S1, S2, S3) и останавливается в ST_CLOSE. Из состояния ST_CLOSE при команде открытия дверь движется по цепочке ST_ACC_NEG, ST_NEG, ST_DEC_NEG (переходы по датчикам S2, S1, S0) и останавливается в ST_OPEN. Если направление изменено во время движения, автомат переходит на противоположную цепочку, при этом проверка смены направления выполняется раньше проверки датчиков. Полный перечень переходов приведён в таблице 2.

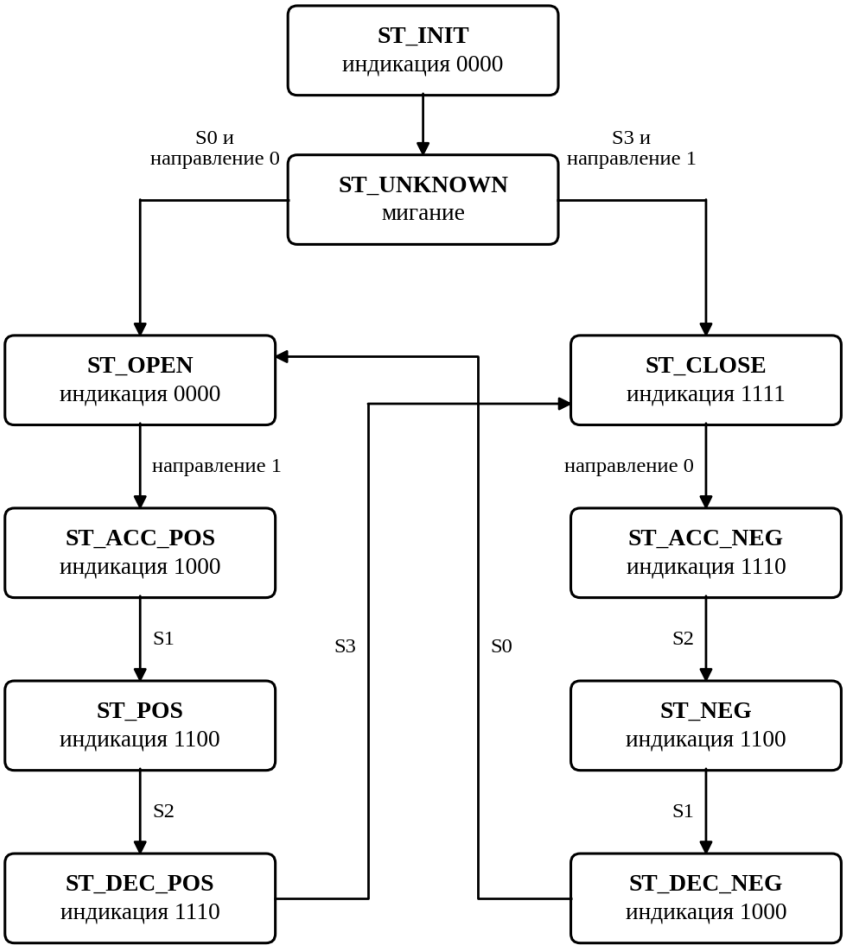


Рис. 2 Структурная схема конечного автомата двери (основные переходы)

Таблица 2 – Переходы конечного автомата двери

Состояние	**Условие перехода**	**Следующее состояние**
ST_INIT	преобразователь находится в рабочем состоянии	ST_UNKNOWN
ST_UNKNOWN	направление 0 и сработал датчик S0	ST_OPEN
ST_UNKNOWN	направление 1 и сработал датчик S3	ST_CLOSE
ST_OPEN	направление 1	ST_ACC_POS
ST_ACC_POS	сработал датчик S1	ST_POS
ST_ACC_POS	направление 0	ST_DEC_NEG
ST_POS	сработал датчик S2	ST_DEC_POS
ST_POS	направление 0	ST_NEG
ST_DEC_POS	сработал датчик S3	ST_CLOSE
ST_DEC_POS	направление 0	ST_ACC_NEG
ST_CLOSE	направление 0	ST_ACC_NEG

Состояние	Условие перехода	Следующее состояние
ST_ACC_NEG	сработал датчик S2	ST_NEG
ST_ACC_NEG	направление 1	ST_DEC_POS
ST_NEG	сработал датчик S1	ST_DEC_NEG
ST_NEG	направление 1	ST_POS
ST_DEC_NEG	сработал датчик S0	ST_OPEN
ST_DEC_NEG	направление 1	ST_ACC_POS

Если преобразователь выходит из рабочего состояния, автомат двери возвращается в ST_INIT и после восстановления рабочего состояния снова определяет положение двери.

Управление частотным преобразователем также реализовано конечным автоматом (рисунок 3). В исходном состоянии выдаётся команда останова 0x06. Когда слово состояния преобразователя после наложения маски 0x6F принимает значение 0x21 (готовность к включению), автомат переходит к команде включения 0x07. При значении 0x23 (преобразователь включён) выдаётся команда разрешения работы 0x0F, и при значении 0x27 преобразователь отрабатывает заданную скорость. Если слово состояния не соответствует ожидаемому либо снято разрешение работы блока, снова выдаётся команда останова. Значения кодов команд и слова состояния приняты по стандартному профилю управления приводом; в эксперименте подтверждены команда 0x0F (15) и значение 0x27 после наложения маски (подраздел 4.7).

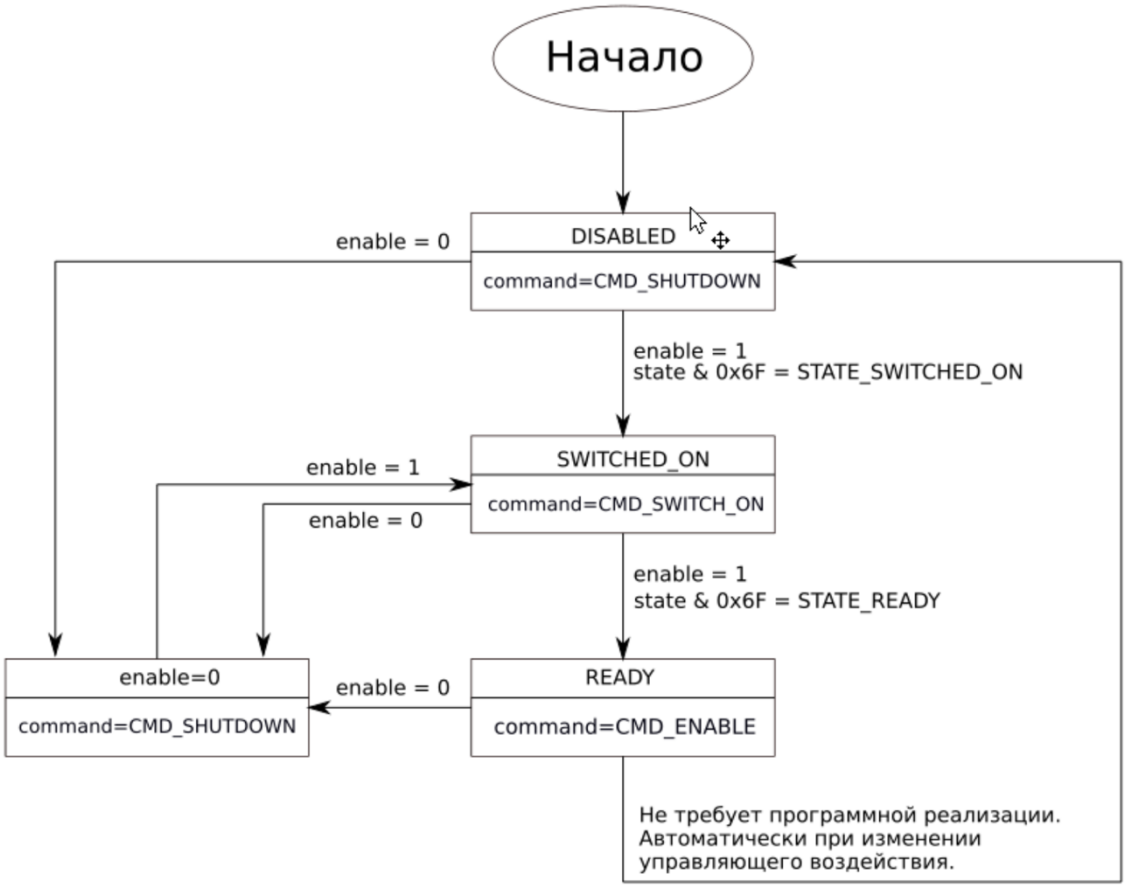


Рис. 3 Блок-схема управления частотным преобразователем

4.3 Описание состояний

Для каждого состояния двери в таблице 3 указаны числовой код, по которому состояние передаётся блоку индикации, вид индикации (единица означает горящий светодиод, порядок записи от первого светодиода к четвёртому), модуль заданной скорости и назначение состояния. Коды соответствуют порядку состояний: от 0 для ST_INIT до 9 для ST_DEC_NEG. Окончания POS и NEG в названиях состояний обозначают цепочки движения: POS соответствует движению от датчика S0 к датчику S3 (закрытие), NEG движению от S3 к S0 (открытие).

Таблица 3 – Состояния двери

Состояние	Код	Индикация	Скорость, об/мин	Описание
ST_INIT	0	0000	0	Инициализация, ожидание перехода преобразователя в рабочее состояние
ST_UNKNOWN	1	мигание	200	Положение неизвестно, движение с малой скоростью до датчика конечного положения
ST_OPEN	2	0000	0	Дверь открыта, на датчике S0
ST_CLOSE	3	1111	0	Дверь закрыта, на датчике S3
ST_ACC_POS	4	1000	200	Разгон при закрытии, участок S0–S1
ST_ACC_NEG	5	1110	200	Разгон при открытии, участок S3–S2
ST_POS	6	1100	850	Движение с максимальной скоростью при закрытии, участок S1–S2
ST_NEG	7	1100	850	Движение с максимальной скоростью при открытии, участок S2–S1
ST_DEC_POS	8	1110	200	Замедление при закрытии перед датчиком S3
ST_DEC_NEG	9	1000	200	Замедление при открытии перед датчиком S0

4.4 Описание функциональных блоков

Система построена из трёх функциональных блоков в библиотеке Library. Каждый блок создан один раз и используется в программе как экземпляр, что позволяет применять блоки в других проектах без изменения кода. Объявления блоков показаны на рисунке 4.

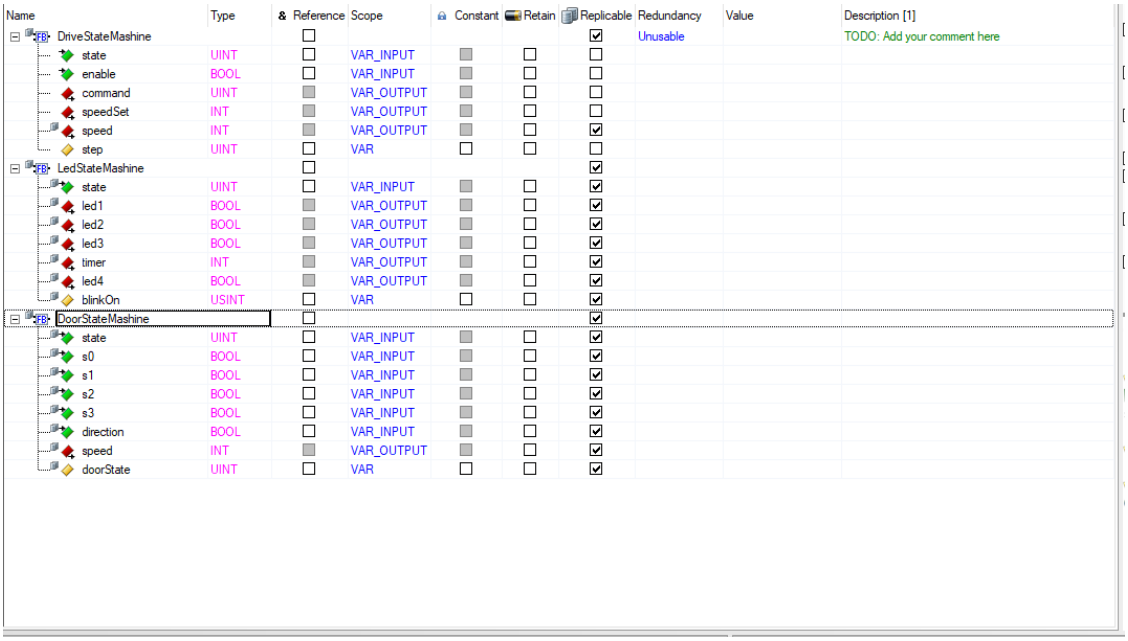


Рис. 4 Объявление функциональных блоков в библиотеке

Блок DriveStateMashine управляет частотным преобразователем по автомату на рисунке 3: по слову состояния и сигналу разрешения формирует команду управления и скорость для преобразователя (таблица 4).

Таблица 4 – Конфигурация функционального блока DriveStateMashine

Конфигурация	**Имя**	**Тип данных**	**Описание**
вход	state	UINT	слово состояния частотного преобразователя
вход	enable	BOOL	сигнал разрешения работы блока
выход	command	UINT	команда управления, подаваемая на преобразователь
выход	speedSet	INT	заданная скорость, передаваемая на преобразователь, об/мин
выход	speed	INT	скорость, формируемая блоком с учётом состояния преобразователя, об/мин
внутр. состояние	step	UINT	служебная переменная шага автомата

Блок LedStateMashine отвечает за светодиодные индикаторы (таблица 5). По коду состояния двери он выставляет выходы индикации в соответствии с таблицей 3, а в состоянии ST_UNKNOWN формирует мигание с помощью счётчика циклов и признака фазы мигания.

Таблица 5 – Конфигурация функционального блока LedStateMashine

Конфигурация	**Имя**	**Тип данных**	**Описание**
вход	state	UINT	код состояния двери
выход	led1, led2, led3, led4	BOOL	сигналы светодиодных индикаторов
выход	timer	INT	счётчик циклов для отсчёта полупериода мигания
внутр. состояние	blinkOn	USINT	фаза мигания (1 индикаторы включены)

Блок DoorStateMashine реализует основную логику: по слову состояния преобразователя, сигналам датчиков и команде направления он определяет текущее состояние двери и задаёт скорость движения (таблица 6). Для остановленной двери скорость равна нулю, на участках S0–S1 и S2–S3 задаётся малая скорость, на участке S1–S2 максимальная.

Таблица 6 – Конфигурация функционального блока DoorStateMashine

Конфигурация	**Имя**	**Тип данных**	**Описание**
вход	state	UINT	слово состояния частотного преобразователя
вход	s0, s1, s2, s3	BOOL	сигналы датчиков положения
вход	direction	BOOL	команда направления движения (тумблер)
выход	speed	INT	заданная скорость, об/мин
внутр. состояние	doorState	UINT	код текущего состояния двери

4.5 Описание структуры программного обеспечения

Программа Main содержит циклическую подпрограмму, в которой вызываются экземпляры трёх блоков. Объявление экземпляров показано на рисунке 5, структура взаимодействия блоков на рисунке 6.

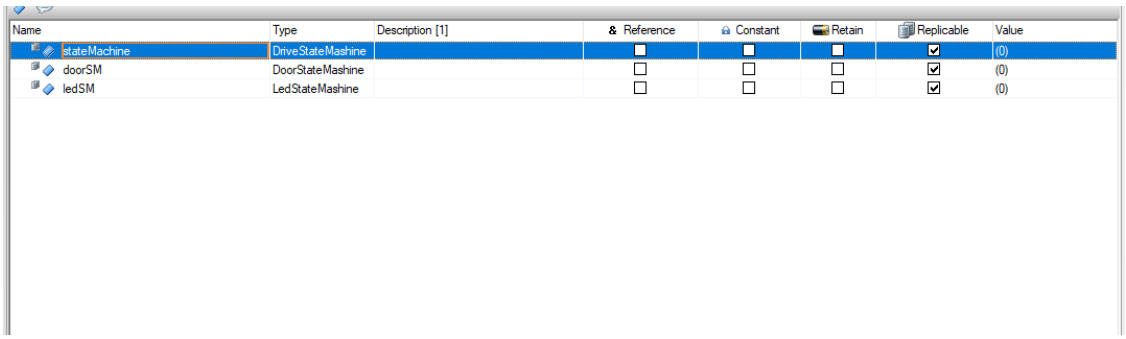


Рис. 5 Экземпляры функциональных блоков в программе Main

В каждом цикле сигнал разрешения работы привода включается постоянно, а слово состояния преобразователя, поступающее по мэппингу, передаётся блоку двери. После вызова блока двери его заданная скорость передаётся блоку управления преобразователем, который формирует команду и скорость для преобразователя. Код текущего состояния двери передаётся блоку индикации. Сигналы датчиков и тумблера поступают на блок двери непосредственно по мэппингу, а выходы блока индикации и блока управления преобразователем передаются на модуль выходов и на преобразователь также по мэппингу.

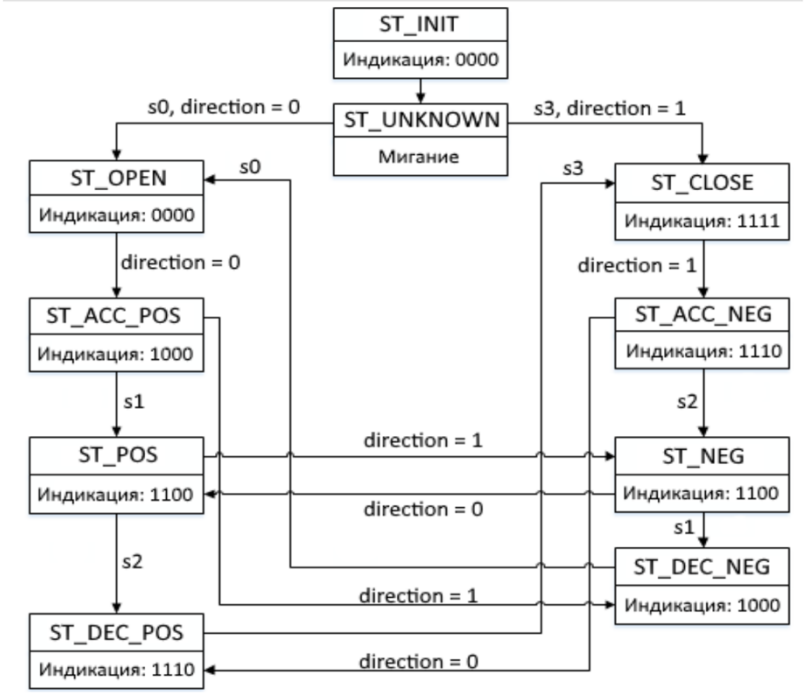


Рис. 6 Блок-схема состояний ворот и возможных переходов между ними.

Мигание индикаторов с частотой 1 Гц формируется счётчиком циклов. Для периода 1 с индикаторы должны быть включены 0,5 с и выключены 0,5 с, поэтому фаза мигания переключается через число циклов, определяемое по формуле (2):

$$N = \frac{T}{2\,T_{\text{ц}}} = \frac{1000}{2 \cdot 10} = 50 \tag{2}$$

где N – число циклов между переключениями фазы мигания; T – период мигания, равный 1000 мс; $T_{\text{ц}}$ – время цикла задачи, принятое равным 10 мс.

4.6 Мэппинг переменных

Для совмещения работы аппаратной и программной частей выполнен мэппинг переменных на каналы преобразователя (таблица 7), датчиков и тумблера (таблица 8) и индикаторов (таблица 9).

Таблица 7 – Мэппинг переменных для частотного преобразователя

Регистр	Переменная	Назначение
ETAD_Input	stateMachine.state	слово состояния преобразователя
CMDD_Output	stateMachine.command	команда управления
LFRD_Output	stateMachine.speedSet	заданное значение скорости

Таблица 8 – Мэппинг переменных для датчиков положения и тумблера (модуль X20DI9371)

Канал	Переменная
DigitalInput01	doorSM.direction
DigitalInput02	doorSM.s0
DigitalInput03	doorSM.s1
DigitalInput04	doorSM.s2
DigitalInput05	doorSM.s3

Таблица 9 – Мэппинг переменных для индикаторов (модуль X20DO9322)

Канал	Переменная
DigitalOutput02	ledSM.led1
DigitalOutput03	ledSM.led2
DigitalOutput04	ledSM.led3

Канал	Переменная
DigitalOutput05	ledSM.led4

4.7 Результаты экспериментальных исследований

Работоспособность системы проверялась в режиме онлайн в окне Watch среды Automation Studio. Для проверки реакции блоков состояния двери последовательно задавались записью кода в окне Watch. Состояния ST_UNKNOWN, ST_CLOSE, ST_ACC_POS, ST_POS и ST_DEC_POS зафиксированы при положении тумблера направления 1, состояния ST_OPEN, ST_ACC_NEG, ST_NEG и ST_DEC_NEG при положении тумблера 0, датчики во всех случаях оставались в нулевом состоянии. Снимки приведены на рисунках 7–15, результаты сведены в таблицу 10.

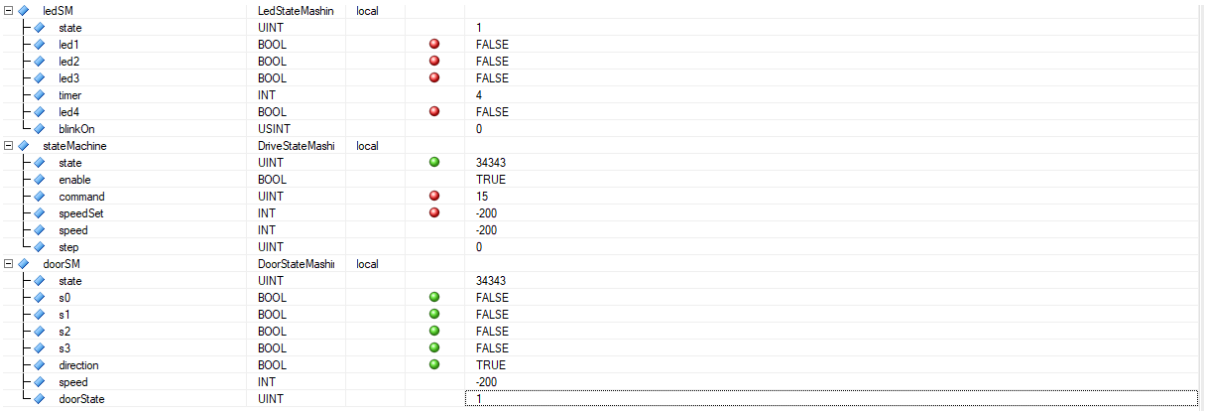


Рис. 7 Окно Watch: состояние ST_UNKNOWN (код 1)

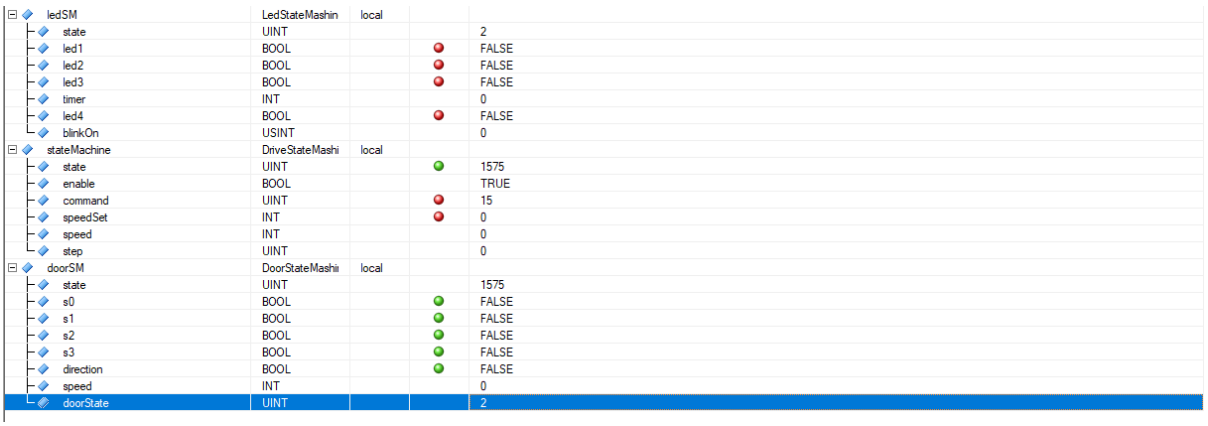
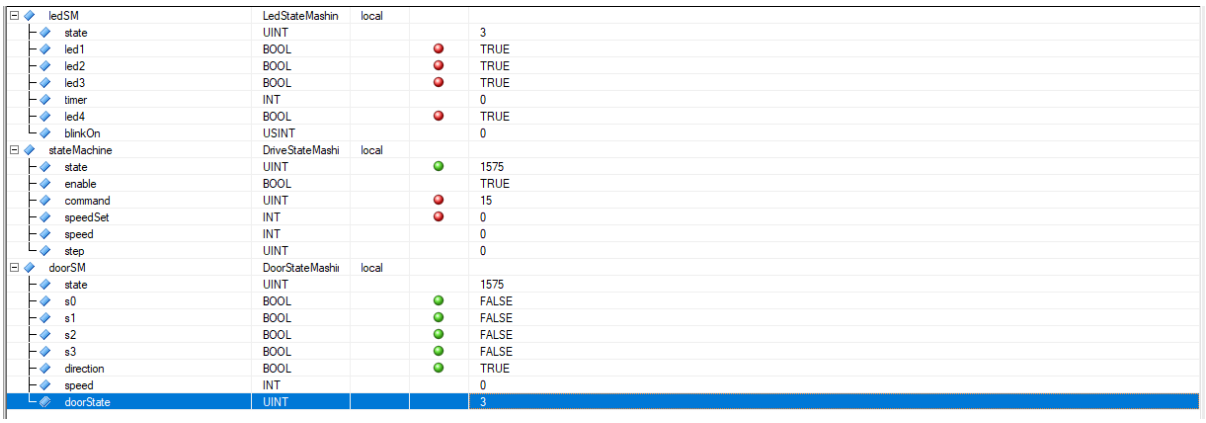
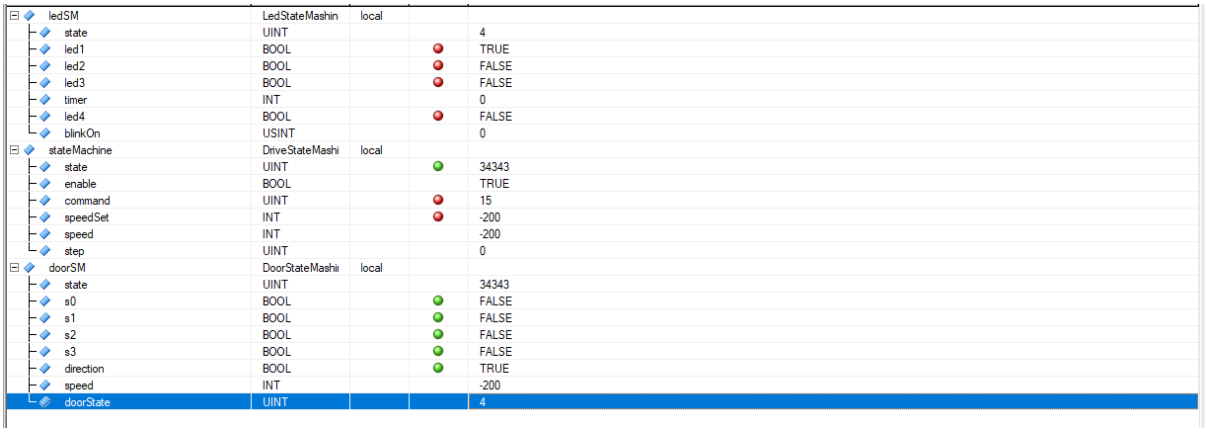


Рис. 8 Окно Watch: состояние ST_OPEN (код 2)



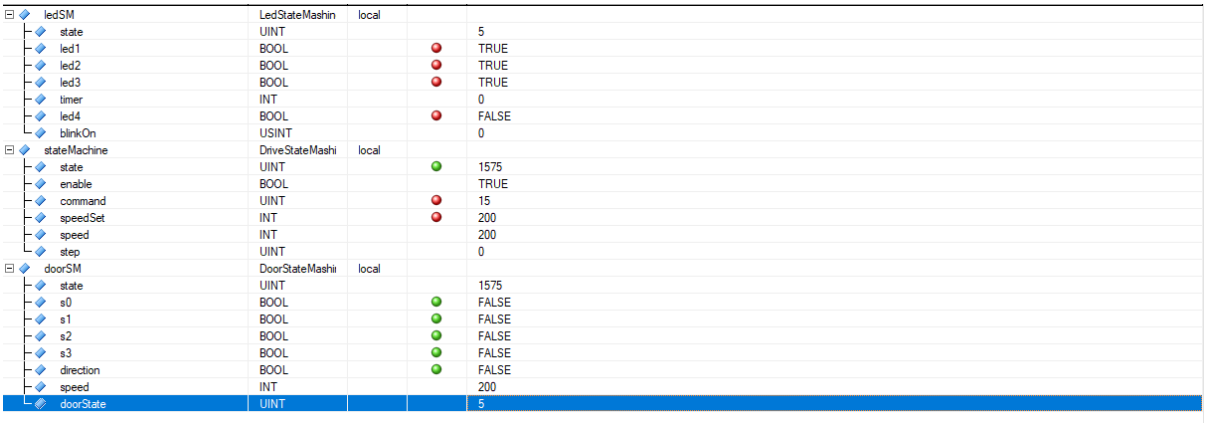
ledSM	LedStateMashin	local		
state	UINT			3
led1	BOOL			TRUE
led2	BOOL			TRUE
led3	BOOL			TRUE
timer	INT			0
led4	BOOL			TRUE
blinkOn	USINT			0
stateMachine	DriveStateMashi	local		
state	UINT			1575
enable	BOOL			TRUE
command	UINT			15
speedSet	INT			0
speed	INT			0
step	UINT			0
doorSM	DoorStateMashii	local		
state	UINT			1575
s0	BOOL			FALSE
s1	BOOL			FALSE
s2	BOOL			FALSE
s3	BOOL			FALSE
direction	BOOL			TRUE
speed	INT			0
doorState	UINT			3

Рис. 9 Окно Watch: состояние ST_CLOSE (код 3)



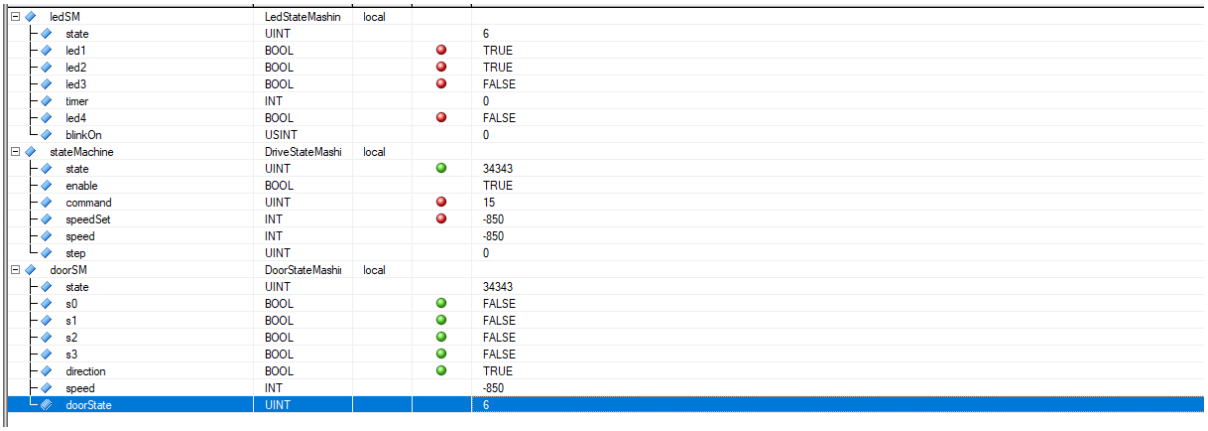
ledSM	LedStateMashin	local		
state	UINT			4
led1	BOOL			TRUE
led2	BOOL			FALSE
led3	BOOL			FALSE
timer	INT			0
led4	BOOL			FALSE
blinkOn	USINT			0
stateMachine	DriveStateMashi	local		
state	UINT			34343
enable	BOOL			TRUE
command	UINT			15
speedSet	INT			-200
speed	INT			-200
step	UINT			0
doorSM	DoorStateMashii	local		
state	UINT			34343
s0	BOOL			FALSE
s1	BOOL			FALSE
s2	BOOL			FALSE
s3	BOOL			FALSE
direction	BOOL			TRUE
speed	INT			-200
doorState	UINT			4

Рис. 10 Окно Watch: состояние ST_ACC_POS (код 4)



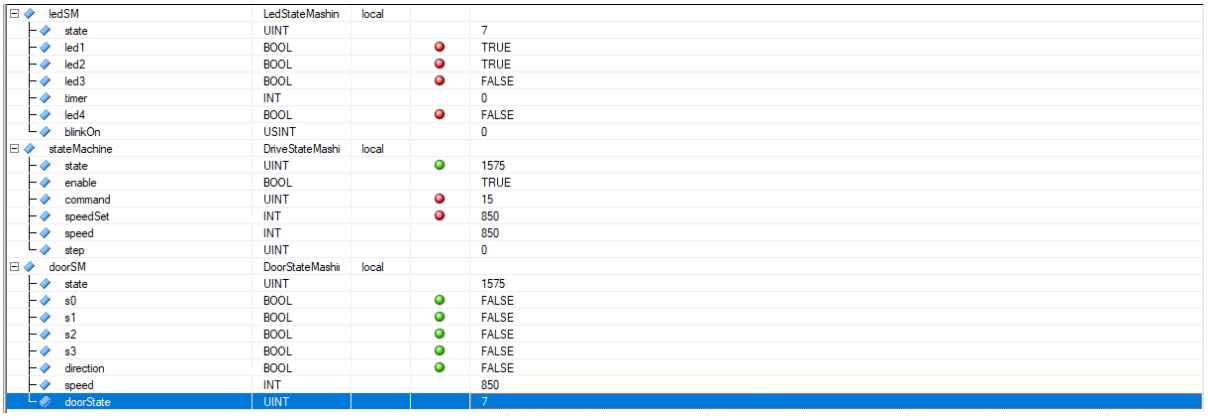
ledSM	LedStateMashin	local		
state	UINT			5
led1	BOOL			TRUE
led2	BOOL			TRUE
led3	BOOL			TRUE
timer	INT			0
led4	BOOL			FALSE
blinkOn	USINT			0
stateMachine	DriveStateMashi	local		
state	UINT			1575
enable	BOOL			TRUE
command	UINT			15
speedSet	INT			200
speed	INT			200
step	UINT			0
doorSM	DoorStateMashii	local		
state	UINT			1575
s0	BOOL			FALSE
s1	BOOL			FALSE
s2	BOOL			FALSE
s3	BOOL			FALSE
direction	BOOL			FALSE
speed	INT			200
doorState	UINT			5

Рис. 11 Окно Watch: состояние ST_ACC_NEG (код 5)



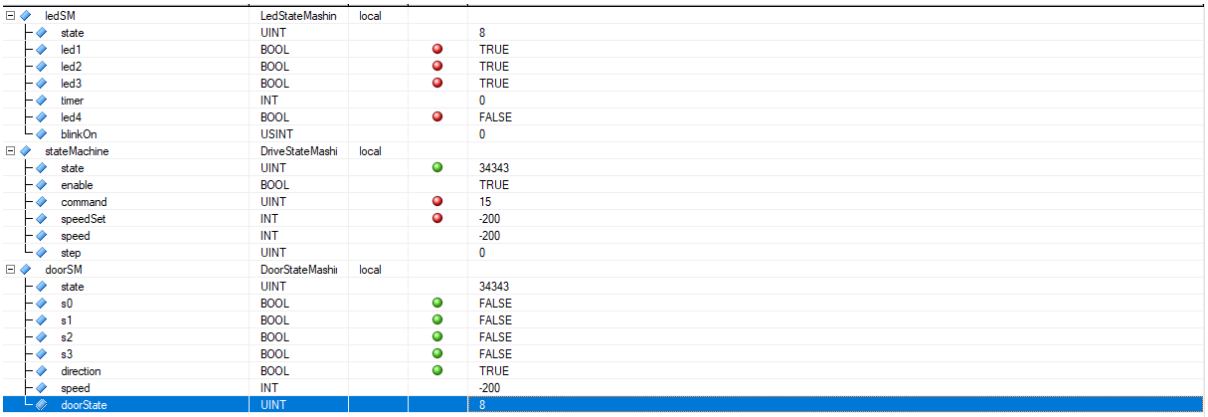
ledSM	LedStateMashin	local		
state	UINT			6
led1	BOOL			TRUE
led2	BOOL			TRUE
led3	BOOL			FALSE
timer	INT			0
led4	BOOL			FALSE
blinkOn	USINT			0
stateMachine	DriveStateMashi	local		
state	UINT			34343
enable	BOOL			TRUE
command	UINT			15
speedSet	INT			-850
speed	INT			-850
step	UINT			0
doorSM	DoorStateMashii	local		
state	UINT			34343
s0	BOOL			FALSE
s1	BOOL			FALSE
s2	BOOL			FALSE
s3	BOOL			FALSE
direction	BOOL			TRUE
speed	INT			-850
doorState	UINT			6

Рис. 12 Окно Watch: состояние ST_POS (код 6)



ledSM	LedStateMashin	local		
state	UINT			7
led1	BOOL			TRUE
led2	BOOL			TRUE
led3	BOOL			FALSE
timer	INT			0
led4	BOOL			FALSE
blinkOn	USINT			0
stateMachine	DriveStateMashi	local		
state	UINT			1575
enable	BOOL			TRUE
command	UINT			15
speedSet	INT			850
speed	INT			850
step	UINT			0
doorSM	DoorStateMashii	local		
state	UINT			1575
s0	BOOL			FALSE
s1	BOOL			FALSE
s2	BOOL			FALSE
s3	BOOL			FALSE
direction	BOOL			FALSE
speed	INT			850
doorState	UINT			7

Рис. 13 Окно Watch: состояние ST_NEG (код 7)



ledSM	LedStateMashin	local		
state	UINT			8
led1	BOOL			TRUE
led2	BOOL			TRUE
led3	BOOL			TRUE
timer	INT			0
led4	BOOL			FALSE
blinkOn	USINT			0
stateMachine	DriveStateMashi	local		
state	UINT			34343
enable	BOOL			TRUE
command	UINT			15
speedSet	INT			-200
speed	INT			-200
step	UINT			0
doorSM	DoorStateMashii	local		
state	UINT			34343
s0	BOOL			FALSE
s1	BOOL			FALSE
s2	BOOL			FALSE
s3	BOOL			FALSE
direction	BOOL			TRUE
speed	INT			-200
doorState	UINT			8

Рис. 14 Окно Watch: состояние ST_DEC_POS (код 8)

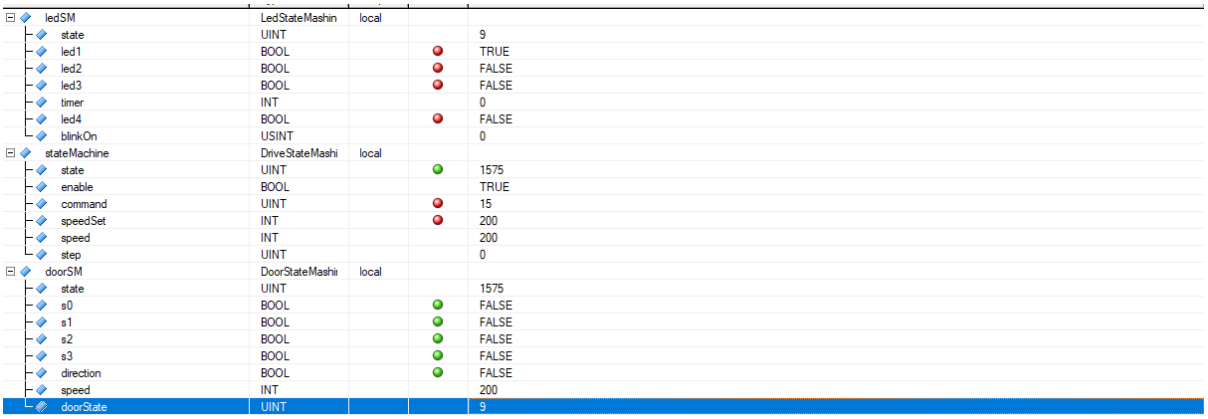


Рис. 15 Окно Watch: состояние ST_DEC_NEG (код 9)

Таблица 10 – Результаты эксперимента

Рис.	Состояние (код)	Направление	Скорость, об/мин	Индикация	Слово состояния
7	ST_UNKNOWN (1)	1	-200	0000 (фаза выключения при мигании)	34343
8	ST_OPEN (2)	0	0	0000	1575
9	ST_CLOSE (3)	1	0	1111	1575
10	ST_ACC_POS (4)	1	-200	1000	34343
11	ST_ACC_NEG (5)	0	200	1110	1575
12	ST_POS (6)	1	-850	1100	34343
13	ST_NEG (7)	0	850	1100	1575
14	ST_DEC_POS (8)	1	-200	1110	34343
15	ST_DEC_NEG (9)	0	200	1000	1575

Индикация во всех проверенных состояниях совпадает с таблицей 3: в ST_OPEN светодиоды погашены, в ST_ACC_POS и ST_DEC_NEG горит один светодиод, в ST_POS и ST_NEG два, в ST_ACC_NEG и ST_DEC_POS три, в ST_CLOSE все четыре. В состоянии ST_UNKNOWN индикаторы мигают: в момент снимка счётчик циклов равен 4, фаза мигания выключена, все светодиоды погашены. Заданная скорость соответствует состоянию: 0 в ST_OPEN и ST_CLOSE, малая скорость 200 об/мин по модулю в ST_UNKNOWN, ST_ACC_POS, ST_ACC_NEG, ST_DEC_POS и

ST_DEC_NEG, максимальная скорость 850 об/мин по модулю в ST_POS и ST_NEG.

Знак заданной скорости определяется положением тумблера направления: при значении 1 скорость отрицательная (-200 об/мин в ST_UNKNOWN, ST_ACC_POS и ST_DEC_POS, -850 об/мин в ST_POS, ноль в ST_CLOSE), при значении 0 положительная (200 об/мин в ST_ACC_NEG и ST_DEC_NEG, 850 об/мин в ST_NEG, ноль в ST_OPEN), что соответствует алгоритму блока двери.

Во всех снимках блок управления преобразователем находится в рабочем режиме: разрешение работы включено, команда равна 15, что соответствует команде разрешения работы 0x0F. Слово состояния преобразователя равно 1575 (0x0627) при нулевой и положительной скорости и 34343 (0x8627) при отрицательной скорости. Во всех случаях после наложения маски 0x6F получается 0x27, то есть работа преобразователя разрешена, а различие в старшем бите совпадает со сменой знака скорости: в конфигурации регистров преобразователя этот бит слова состояния обозначает направление вращения (значение 0 соответствует прямому вращению). Заданная скорость, передаваемая на преобразователь, совпадает со скоростью, сформированной блоком двери, слово состояния в блоке двери равно слову состояния преобразователя, а код состояния в блоке индикации равен коду состояния двери, что подтверждает связи между блоками, описанные в подразделе 4.5.

5 Вывод

В ходе выполнения лабораторной работы разработано и проверено на учебном стенде программное обеспечение системы управления приводом защитной двери на языке ANSI C в среде B&R Automation Studio версии 4.5.2.102. Логика построена на конечных автоматах и разделена на три повторно используемых функциональных блока: управления частотным преобразователем (три шага: останов, включение, разрешение работы),

светодиодной индикации и управления дверью (десять состояний с переходами по сигналам датчиков положения и по смене направления).

Выполнена настройка частотного преобразователя по паспортным данным двигателя, регистры состояния и управления переведены из битового представления в представление словами, выполнен мэппинг переменных на каналы датчиков, тумблера направления, индикаторов и регистры преобразователя. Проект собран без ошибок, загружен на контроллер и проверен в режиме онлайн.

В эксперименте проверены девять состояний двери из десяти: от ST_UNKNOWN до ST_DEC_NEG, кроме начального состояния ST_INIT. Для каждого состояния получены индикация и заданная скорость, и они совпали с таблицей 3. Скорость 200 об/мин (22,5 % от номинальной) задавалась в режиме определения положения, при разгоне и при замедлении, скорость 850 об/мин (95,5 % от номинальной) на участке между датчиками S1 и S2, в открытом и закрытом положениях скорость равна нулю. Индикация 0000, 1000, 1100, 1110 и 1111 нарастает по мере движения к закрытому положению, в режиме определения положения индикаторы мигают. Преобразователь во всех снимках находился в рабочем режиме (команда 15, младший байт слова состояния 0x27), заданная скорость блока двери совпадала со скоростью, передаваемой на преобразователь, а код состояния двери совпадал с кодом на входе блока индикации, что подтверждает правильность связей между блоками. Знак заданной скорости определялся положением тумблера направления: положительный при значении 0 и отрицательный при значении 1, одновременно менялся старший бит слова состояния преобразователя (1575 при нулевой и положительной скорости, 34343 при отрицательной), что соответствует смене направления вращения.

Состояния задавались записью кода в окне Watch при неподвижных датчиках, поэтому полный цикл движения двери с переходами по датчикам и реверс на ходу в данной работе не снимались, а частота мигания 1 Гц обеспечивается расчётом числа циклов и по снимкам не измерялась. Проверка

этих режимов остаётся предметом дальнейших испытаний. Разделение системы на отдельные блоки упрощает отладку, поскольку каждый блок можно проверять отдельно, и позволяет переносить блоки в другие проекты без изменения кода.

ПРИЛОЖЕНИЕ А

Листинг программы

Листинг файла Cyclic.c (программа Main)

```
#include <bur/plctypes.h>
#ifdef _DEFAULT_INCLUDES
#include <AsDefault.h>
#endif

void _CYCLIC ProgramCyclic(void)
{
    stateMachine.enable = 1;

    doorSM.state = stateMachine.state;
    DoorStateMashine(&doorSM);

    stateMachine.speed = doorSM.speed;
    stateMachine.speedSet = doorSM.speed;
    DriveStateMashine(&stateMachine);

    ledSM.state = doorSM.doorState;
    LedStateMashine(&ledSM);
}
```

Листинг файла DriveStateMashine.c

```
#include <bur/plctypes.h>
#ifdef _DEFAULT_INCLUDES
#include <AsDefault.h>
#endif

#define CMD_SHUTDOWN 0x0006
#define CMD_SWITCH_ON 0x0007
#define CMD_ENABLE 0x000F
#define STATE_MASK 0x006F
#define STATE_SWITCHED_ON 0x0021
#define STATE_READY 0x0023
#define STATE_OPERATION 0x0027

void DriveStateMashine(struct DriveStateMashine* inst)
{
    UINT st = inst->state & STATE_MASK;

    if (!inst->enable) {
        inst->command = CMD_SHUTDOWN;
        inst->speed = 0;
        return;
    }

    switch (inst->command) {

    case CMD_SWITCH_ON:
        inst->speed = 0;
        if (st == STATE_READY) inst->command = CMD_ENABLE;
        else if (st != STATE_SWITCHED_ON) inst->command = CMD_SHUTDOWN;
        break;

    case CMD_ENABLE:
        inst->speed = (st == STATE_OPERATION) ? inst->speedSet : 0;
```

```
        if (st != STATE_READY && st != STATE_OPERATION) inst->command =
CMD_SHUTDOWN;
        break;

    default:
        inst->command = CMD_SHUTDOWN;
        inst->speed = 0;
        if (st == STATE_SWITCHED_ON) inst->command = CMD_SWITCH_ON;
        break;
    }
}
```

Листинг файла LedStateMashine.c

```
#include <bur/plctypes.h>
#ifdef _DEFAULT_INCLUDES
#include <AsDefault.h>
#endif

#define ST_INIT 0
#define ST_UNKNOWN 1
#define ST_OPEN 2
#define ST_CLOSE 3
#define ST_ACC_POS 4
#define ST_ACC_NEG 5
#define ST_POS 6
#define ST_NEG 7
#define ST_DEC_POS 8
#define ST_DEC_NEG 9

#define CYCLE_MS 10
#define HALF_CYCLES (500 / CYCLE_MS)

void LedStateMashine(struct LedStateMashine* inst)
{
    if (inst->state == ST_UNKNOWN) {
        inst->timer++;
        if (inst->timer >= HALF_CYCLES) {
            inst->timer = 0;
            inst->blinkOn = !inst->blinkOn;
        }
    } else {
        inst->timer = 0;
        inst->blinkOn = 0;
    }

    switch (inst->state) {

    case ST_UNKNOWN:
        inst->led1 = inst->blinkOn; inst->led2 = inst->blinkOn;
        inst->led3 = inst->blinkOn; inst->led4 = inst->blinkOn;
        break;

    case ST_ACC_POS:
    case ST_DEC_NEG:
        inst->led1 = 1; inst->led2 = 0; inst->led3 = 0; inst->led4 = 0;
        break;

    case ST_POS:
    case ST_NEG:
        inst->led1 = 1; inst->led2 = 1; inst->led3 = 0; inst->led4 = 0;
        break;

    case ST_ACC_NEG:
```

```
    case ST_DEC_POS:
        inst->led1 = 1; inst->led2 = 1; inst->led3 = 1; inst->led4 = 0;
        break;

    case ST_CLOSE:
        inst->led1 = 1; inst->led2 = 1; inst->led3 = 1; inst->led4 = 1;
        break;

    default:
        inst->led1 = 0; inst->led2 = 0; inst->led3 = 0; inst->led4 = 0;
        break;
    }
}
```

Листинг файла DoorStateMashine.c

```
#include <bur/plctypes.h>
#ifdef _DEFAULT_INCLUDES
#include <AsDefault.h>
#endif

#define ST_INIT 0
#define ST_UNKNOWN 1
#define ST_OPEN 2
#define ST_CLOSE 3
#define ST_ACC_POS 4
#define ST_ACC_NEG 5
#define ST_POS 6
#define ST_NEG 7
#define ST_DEC_POS 8
#define ST_DEC_NEG 9

#define SPEED_SLOW 200
#define SPEED_FAST 850
#define STATE_MASK 0x006F
#define STATE_OPERATION 0x0027

void DoorStateMashine(struct DoorStateMashine* inst)
{
    INT mag = 0;

    if ((inst->state & STATE_MASK) != STATE_OPERATION) {
        inst->doorState = ST_INIT;
        inst->speed = 0;
        return;
    }

    switch (inst->doorState) {

    case ST_INIT:
        inst->doorState = ST_UNKNOWN;
        break;

    case ST_UNKNOWN:
        if (!inst->direction && inst->s0) inst->doorState = ST_OPEN;
        else if (inst->direction && inst->s3) inst->doorState = ST_CLOSE;
        break;

    case ST_OPEN:
        if (inst->direction) inst->doorState = ST_ACC_POS;
        break;

    case ST_ACC_POS:
        if (!inst->direction) inst->doorState = ST_DEC_NEG;
```

```
            else if (inst->s1) inst->doorState = ST_POS;
            break;

        case ST_POS:
            if (!inst->direction) inst->doorState = ST_NEG;
            else if (inst->s2) inst->doorState = ST_DEC_POS;
            break;

        case ST_DEC_POS:
            if (!inst->direction) inst->doorState = ST_ACC_NEG;
            else if (inst->s3) inst->doorState = ST_CLOSE;
            break;

        case ST_CLOSE:
            if (!inst->direction) inst->doorState = ST_ACC_NEG;
            break;

        case ST_ACC_NEG:
            if (inst->direction) inst->doorState = ST_DEC_POS;
            else if (inst->s2) inst->doorState = ST_NEG;
            break;

        case ST_NEG:
            if (inst->direction) inst->doorState = ST_POS;
            else if (inst->s1) inst->doorState = ST_DEC_NEG;
            break;

        case ST_DEC_NEG:
            if (inst->direction) inst->doorState = ST_ACC_POS;
            else if (inst->s0) inst->doorState = ST_OPEN;
            break;

        default:
            inst->doorState = ST_INIT;
            break;
        }

        switch (inst->doorState) {
        case ST_UNKNOWN:
        case ST_ACC_POS:
        case ST_ACC_NEG:
        case ST_DEC_POS:
        case ST_DEC_NEG:
            mag = SPEED_SLOW;
            break;
        case ST_POS:
        case ST_NEG:
            mag = SPEED_FAST;
            break;
        default:
            mag = 0;
            break;
        }

        inst->speed = inst->direction ? -mag : mag;
    }
```